

# UNIX and Linux Complete Study Material

---

## UNIT 1

### 1. Introduction to UNIX

---

UNIX is a powerful, multiuser, multitasking operating system developed in 1969 at Bell Labs by Ken Thompson, Dennis Ritchie, and their team. It was originally designed for programmers and researchers who needed a reliable and efficient computing environment. UNIX became popular because of its portability, stability, security, and simple design philosophy.

The main idea behind UNIX is that every task should be performed by a small program that does one job well. Users can combine these programs to perform complex tasks. UNIX supports multiple users working simultaneously and multiple processes running at the same time. It provides a hierarchical file system, powerful command-line interface, process management, memory management, and networking capabilities.

UNIX has influenced many modern operating systems, including Linux, macOS, Solaris, and BSD. Today, UNIX concepts form the foundation of most modern operating systems. Because of its reliability and efficiency, UNIX is widely used in servers, workstations, cloud computing, and enterprise environments.

### 2. Discuss About POSIX

---

POSIX stands for Portable Operating System Interface. It is a set of standards developed by the IEEE to ensure compatibility between UNIX-like operating systems.

Before POSIX, different UNIX versions had different commands, system calls, and interfaces, making software portability difficult. POSIX solved this problem by defining a common standard that operating systems should follow. If software is developed according to POSIX standards, it can run on different POSIX-compliant systems with little or no modification.

POSIX specifies standards for command-line shells, file systems, process management, system calls, utilities, and programming interfaces. It includes standards for functions such as file handling, process creation, signals, and thread management.

Examples of POSIX-compliant operating systems include Linux, macOS, Solaris, AIX, and BSD systems. POSIX greatly simplifies software development because programmers can write applications once and run them on many UNIX-like systems. It improves portability, interoperability, and consistency across operating systems.

### 3. Discuss About Linux

---

Linux is a free and open-source operating system based on UNIX principles. It was created in 1991 by Linus Torvalds while he was a student in Finland. Linux uses the Linux kernel along with various GNU utilities to form a complete operating system.

Unlike traditional UNIX systems, Linux source code is publicly available. Anyone can view, modify, and distribute it under open-source licenses. This has made Linux one of the most widely used operating systems in the world.

Linux supports multitasking, multiuser operations, networking, security, process management, memory management, and device management. It is highly stable and secure, making it suitable for servers, cloud platforms, supercomputers, embedded systems, and mobile devices.

Linux is widely used in web servers, data centers, Android smartphones, IoT devices, and enterprise systems. Popular software development tools and programming environments are readily available on Linux. Due to its

flexibility, reliability, and cost-effectiveness, Linux has become one of the most important operating systems in modern computing.

## 4. Most Popular Distributions of Linux

A Linux distribution (distro) is a complete operating system built around the Linux kernel. Different distributions are designed for different purposes and users.

- **Ubuntu:** Developed by Canonical Ltd., Ubuntu is one of the most beginner-friendly Linux distributions. It provides excellent community support and regular updates.
- **Fedora:** Sponsored by Red Hat, Fedora offers the latest software technologies and is popular among developers and enthusiasts.
- **Debian:** Debian is known for its stability and reliability. Many other distributions, including Ubuntu, are based on Debian.
- **Red Hat Enterprise Linux (RHEL):** Used primarily in enterprise environments, RHEL provides commercial support, security, and long-term maintenance.
- **CentOS:** Traditionally based on RHEL, CentOS was widely used for servers because of its stability and compatibility.
- **Linux Mint:** Linux Mint focuses on simplicity and ease of use, making it a popular choice for users migrating from Windows.
- **Arch Linux:** Arch Linux is lightweight and highly customizable, preferred by advanced users who want complete control over their systems.

## 5. Compare Between UNIX and Linux

| Feature          | UNIX                                 | Linux                                     |
|------------------|--------------------------------------|-------------------------------------------|
| Origin           | Developed in 1969 at Bell Labs       | Developed in 1991 by Linus Torvalds       |
| Source Code      | Mostly proprietary                   | Open source                               |
| Cost             | Usually commercial and expensive     | Generally free                            |
| Development      | Controlled by vendors                | Community-driven                          |
| Hardware Support | Limited to vendor-supported hardware | Supports a wide range of hardware         |
| Flexibility      | Less customizable                    | Highly customizable                       |
| Popularity       | Common in enterprise systems         | Used in servers, desktops, mobile devices |
| Examples         | AIX, Solaris, HP-UX                  | Ubuntu, Fedora, Debian                    |

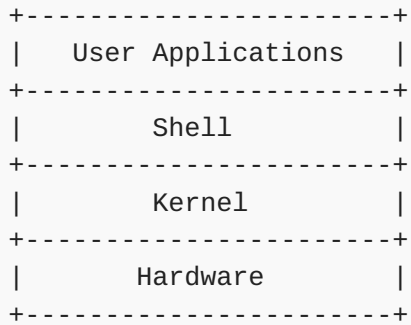
**Summary:** UNIX is the original operating system family, while Linux is a UNIX-like open-source operating system inspired by UNIX principles.

## 6. UNIX System Architecture

UNIX architecture is organized into layers that separate hardware from user applications.

- **Hardware Layer:** The lowest layer consisting of CPU, memory, storage devices, printers, network cards, etc.
- **Kernel Layer:** The core of UNIX. It manages hardware resources and provides services to applications. It handles process, memory, file, and device management.
- **Shell Layer:** The interface between the user and the kernel. Interprets commands. Examples: Bourne Shell (sh), Bash Shell, Korn Shell (ksh), C Shell (csh).
- **Application Layer:** Contains user programs, utilities, editors, compilers, and application software.

## Architecture Diagram:



## 7. Discuss About the UNIX Kernel and System Call Interface

The UNIX Kernel is the central component. It acts as a bridge between hardware and user programs.

### Functions of the Kernel:

- Process Management (Creates, terminates, schedules CPU)
- Memory Management (Allocates/deallocates, virtual memory)
- File System Management (Organizes files, access permissions)
- Device Management (Communicates via device drivers)
- Security and Protection (Authentication and access rights)

### System Call Interface:

A mechanism through which user programs request services from the kernel. Examples: `open()`, `read()`, `write()`, `fork()`, `exec()`, `close()`.

```
User Program -> System Call -> Kernel -> Hardware
```

## 8. UNIX Directory Structure

UNIX uses a hierarchical tree-like directory structure starting from the root directory `/`.

| Directory          | Purpose                        |
|--------------------|--------------------------------|
| <code>/bin</code>  | Essential user commands        |
| <code>/sbin</code> | System administration commands |
| <code>/etc</code>  | Configuration files            |
| <code>/home</code> | User home directories          |
| <code>/root</code> | Home directory of root user    |
| <code>/dev</code>  | Device files                   |
| <code>/proc</code> | Process and kernel information |
| <code>/tmp</code>  | Temporary files                |
| <code>/usr</code>  | User programs and libraries    |

|       |                              |
|-------|------------------------------|
| /var  | Variable data such as logs   |
| /lib  | Shared libraries             |
| /boot | Boot loader and kernel files |

```
/
├─ bin
├─ boot
├─ dev
├─ etc
├─ home
├─ lib
├─ media
├─ mnt
├─ proc
├─ root
├─ sbin
├─ tmp
├─ usr
└─ var
```

## UNIT 2: UNIX File Commands

### 1. Create Directory

---

**Work:** Creates a new directory. **Syntax:** `mkdir directory_name`. **Example:** `mkdir Notes`

### 2. Change Directory

---

**Work:** Changes current working directory. **Example:** `cd Notes`; `cd ..` (Parent); `cd ~` (Home).

### 3. Move and Copy Directory

---

**Move:** `mv Notes Backup` (Renames or moves).

**Copy:** `cp -r Backup Backup_Copy` (-r for recursive).

### 4. Create File

---

**Example:** `touch file1.txt` (Empty file).

**Method 2:** `cat > file1.txt` (Type content, press Ctrl+D).

### 5. Remove File

---

**Work:** Deletes files. **Example:** `rm file1.txt`

### 6. Remove Directory

---

**Empty:** `rmdir Notes`

**Non-Empty:** `rm -r Notes`

### 7. Listing Directory Information

---

**ls:** Shows names.

**ls -l:** Detailed info (Permissions, Owner, Size, Date).

**ls -a:** Shows hidden files (starting with `.`).

### 8. Types of Files

---

- **Regular File:** Stores data (e.g., `notes.txt`)
- **Directory File:** Stores filenames.
- **Character Device:** Data char-by-char (`/dev/tty`).
- **Block Device:** Data in blocks (`/dev/sda`).
- **Symbolic Link:** Shortcut.
- **Socket File/FIFO:** Process communication.

### 9. Change File Security

---

**Example:** `chmod 755 script.sh` (7=rwx, 5=r-x). Owner can read/write/execute, others read/execute.

### 10. Empty File & Timestamp

---

`touch report.txt`: Creates file if missing; updates timestamp if exists.

## 11. Access, Modification, and Change Time

---

- **atime:** Last read (e.g., via `cat`).
- **mtime:** Last content change (e.g., via `echo >>`).
- **ctime:** Last metadata change (e.g., via `chmod`).

View via `stat report.txt`.

## 12. Touch Options

---

**-d:** `touch -d "2025-01-01" file.txt`

**-t:** `touch -t 202501011230 file.txt (YYYYMMDDHHMM)`.

## 13. Finding Files

---

**File:** `find . -name "report.txt"`

**Directory:** `find . -type d -name "Notes"`

## 14. Soft Link and Hard Link

---

**Soft Link:** `ln -s original.txt shortcut.txt` (Pointer; breaks if original deleted).

**Hard Link:** `ln original.txt copy.txt` (Same inode; data persists until all links gone).

## 15. i-Node

---

Metadata store (Owner, Permissions, Size, Disk location). Use `ls -li` to see ID.

## 16. File System Size

---

`df -h`: Displays size, used, and available space.

## 17. Redirection (> and >>)

---

**>:** Overwrite (`echo "Hi" > file.txt`).

**>>:** Append (`echo "Hi" >> file.txt`).

## 18. Pipe and Tee

---

**Pipe (|):** `ls | grep ".txt"` (Output of one becomes input of another).

**Tee:** `ls | tee log.txt` (Saves to file and shows on screen).

## UNIT 3

### 1. Row Wise and Column Wise Selection

---

**Sample file student.txt:**

```
101 Amit 85
102 Rahul 90
103 Priya 88
```

**Row Wise:** `head -2 student.txt` (First 2); `tail -1 student.txt` (Last line).

**Column Wise:** `cut -d " " -f2 student.txt` (Extracts Names).

### 2. Selection Options

---

**Print line 3:** `sed -n '3p' file.txt`

**Multiple Columns:** `cut -d " " -f1,3 student.txt`

### 3. Merge Files Horizontally

---

**paste:** `paste file1.txt file2.txt` (Joins lines side-by-side).

**Delimiter:** `paste -d "," f1 f2`

### 4. Split Large Files

---

`split -l 100 large.txt`: Splits file into smaller ones with 100 lines each (xaa, xab...).

### 5. Counting (wc)

---

`wc -l` (Lines), `wc -w` (Words), `wc -c` (Characters).

### 6 & 7. Sorting

---

**By Column:** `sort -k2 student.txt` (Sort by 2nd field).

**Numerical:** `sort -n numbers.txt`; `sort -nr` (Reverse numeric).

### 8 & 9. Comparison

---

**diff:** Line by line comparison (`diff -y` for side-by-side).

**cmp:** Character/Byte by byte comparison (`cmp -s` for silent mode).

### Quick Exam Summary Table

---

| Command                   | Purpose                      |
|---------------------------|------------------------------|
| <code>mkdir/cd/lis</code> | Directory management         |
| <code>mv/cp/rm</code>     | File/Directory operations    |
| <code>touch</code>        | Create file/Update timestamp |
| <code>chmod</code>        | Security/Permissions         |
| <code>find/ln</code>      | Search and Linking           |
| <code>df -h</code>        | Disk usage                   |

|                   |                            |
|-------------------|----------------------------|
| > / >> /   / tee  | Redirection and Pipes      |
| head / tail / cut | Selection                  |
| paste / split     | Merge/Divide               |
| wc / sort         | Analysis and Alphabetizing |
| diff / cmp        | Comparison                 |